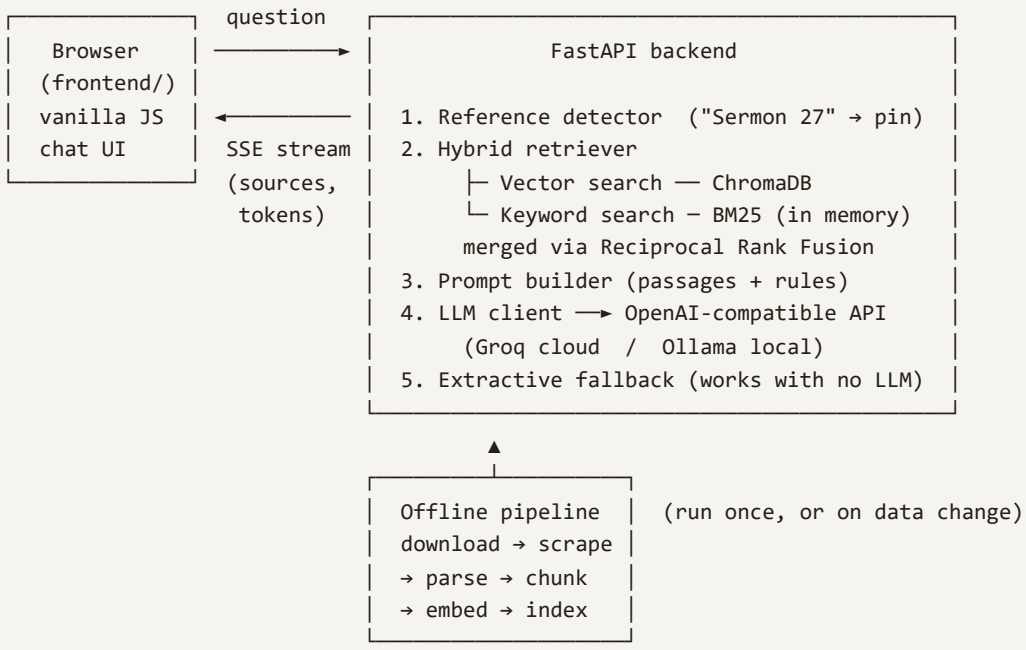
Nahj AI — System Architecture

A production-quality, fully open-source RAG chatbot for Nahjul Balagha
Prepared July 2026 · Repository: github.com/mywebdevworld-pixel/NahjulAI

1. What the system does

Nahj AI answers natural-language questions about **Nahjul Balagha** — the collection of 240 sermons, 79 letters, and 480 sayings of Imam Ali ibn Abi Talib (peace be upon him), compiled by al-Sharif al-Radi (English translation by Sayyid Ali Raza). It uses **Retrieval-Augmented Generation (RAG)**: instead of letting an AI model answer from its own (unreliable) memory, the system first *retrieves* the actual passages relevant to the question, then instructs the model to answer *only* from those passages, citing each one (e.g. `[Sermon 27]`). Every citation is clickable in the UI and opens the full original passage, so every answer is verifiable.

2. High-level architecture



3. Technology stack (all free / open source)

Layer	Technology	Why this choice
Web framework	FastAPI + Uvicorn (Python 3.13)	Async, typed, automatic OpenAPI docs at `/docs`, first-class streaming support
Embeddings	fastembed — model `BAAI/bge-small-en-v1.5` (384-dim)	ONNX runtime: fast on plain CPUs, ~130 MB, no 2.5 GB PyTorch dependency

Vector database	ChromaDB (embedded/persistent mode)	Runs inside the app process — no separate DB server to operate
Keyword search	rank-bm25 (BM25Okapi, in-memory)	Catches exact-word matches that semantic search can miss
LLM	Any OpenAI-compatible endpoint (currently Groq, `llama-3.3-70b-versatile`)	Provider-agnostic: swap Groq / Ollama / OpenRouter by editing 3 env vars
Data validation	Pydantic v2 + pydantic-settings	Typed request/response schemas and .env-driven configuration
Scraping/parsing	httpx, curl, BeautifulSoup + lxml	Robust ingestion of markdown and bilingual HTML sources
Frontend	Vanilla HTML/CSS/JS (zero dependencies)	No build step, no framework churn, dark-mode aware, fully self-contained
Packaging	Docker + docker-compose	One-command reproducible deployment; index baked in at build time
Tests	pytest (11 tests)	Corpus integrity + API behaviour, including the SSE stream

4. The offline data pipeline

Four scripts, run in order, turn public web sources into a searchable knowledge base:

1. **`scripts/download_data.py`** — downloads the sermons/letters markdown (GitHub: MonisBana/Nahjul-Balagha) and the sayings page (al-islam.org) into `data/raw/`. Falls back to `curl` when a host rejects Python's TLS fingerprint.
2. **`scripts/scrape_alislam.py`** — the markdown source was missing 24 letters and 2 sermons, so this scrapes all 79 letters (plus missing sermons) from al-islam.org, caching each page. It filters out Arabic paragraphs (the pages are bilingual) with a script-ratio heuristic.
3. **`scripts/build_corpus.py`** — parses and merges everything into a single normalized file, `data/corpus.json`: 799 documents, each with `{id, type, number, ref, title, text}` (e.g. `sermon-27`, `letter-53`, `saying-100`).
4. **`scripts/ingest.py`** — splits each document into ~1,500-character chunks on paragraph boundaries (with one-paragraph overlap so context isn't cut mid-thought), embeds every chunk into a 384-dimension vector, and stores **1,579 chunks** in ChromaDB with their metadata.

> **Why chunking?** A sermon can be many pages long. Embedding models and LLM context windows work best with focused passages, so long texts are split into overlapping chunks; each chunk remembers which document it came from (`doc_id`, `ref`, `chunk_index`).

5. Where data is stored

Location	Contents	In git?
`data/raw/`	Original downloaded sources (markdown, HTML, cached scrape pages)	No — rebuilt by scripts
`data/corpus.json`	The normalized corpus: 799 documents, ~1.3 M characters of English text	No — rebuilt by scripts
`data/chroma/`	ChromaDB persistent store: an SQLite file + HNSW vector index holding all 1,579 chunk embeddings and metadata	No — rebuilt by ingest
In memory (at startup)	BM25 keyword index + corpus lookup table, built from the two stores above in ~1 second	—
`.env`	Secrets and configuration (LLM endpoint, model, API key)	**Never** — gitignored
Docker image	In deployment, the whole pipeline runs at *build* time, so corpus + index are baked into the image; nothing needs a database server	—

There is no user database: conversation history lives only in the browser tab and is sent with each request (a stateless backend — simple to scale and nothing personal is stored server-side).

6. How a question is answered (request lifecycle)

1. **The browser** POSTs `{message, history}` to `/api/chat`.
2. **Rate limiter** — a per-IP sliding window (10 chats/minute) protects the free LLM quota.
3. **Reference detection** — a regex looks for explicit citations in the question ("letter 31", "hikmah 55"...). Matching documents are fetched directly and *pinned* to the top of the context, guaranteeing "summarize Letter 31" actually sees Letter 31.
4. **Hybrid retrieval** — the question is embedded into a vector and searched in ChromaDB (semantic similarity); simultaneously BM25 ranks chunks by keyword overlap. The two rankings are merged with *Reciprocal Rank Fusion*: each chunk scores `Σ 1/(60 + rank)` across both lists, which rewards passages that both methods agree on. The top 6 chunks survive.
5. **Prompt building** — a system prompt defines the assistant's identity and hard rules (answer only from context, cite every claim as `[Sermon N]`, quote exactly, admit when the context is insufficient, respect the religious significance, mirror the user's language). The retrieved passages and recent conversation turns are appended, then the user's question.
6. **Generation** — the prompt is streamed to the LLM. Tokens are forwarded to the browser in real time over **Server-Sent Events**: first a `sources` event (the retrieved passages, shown as source cards), then `token` events, then `done`.
7. **Fallback** — if no LLM is reachable or the call fails, the API returns the top passages verbatim with citations ("extractive mode") instead of erroring out.

7. The LLM: how we connect and what it is for

Connection

The app speaks the **OpenAI-compatible chat-completions protocol** — a de-facto standard implemented by nearly every LLM provider. The client (`app/rag/generator.py`) is ~50 lines of `httpx` that POSTs to `{LLM_BASE_URL}/chat/completions` with `stream: true` and parses the SSE token stream. Because only three environment variables define the provider, swapping LLMs requires *zero code changes*:

Provider	LLM_BASE_URL	Notes
Groq (current)	`https://api.groq.com/openai/v1`	Hosted, free tier, very fast (Llama 3.3 70B)
Ollama	`http://localhost:11434/v1`	Runs models locally — fully private, offline-capable
OpenRouter	`https://openrouter.ai/api/v1`	Gateway to many free/paid open models

Role of the LLM

The LLM is deliberately given a **narrow job: language, not knowledge**. It does not decide what Nahjul Balagha says — the retrieval layer does that. The LLM's job is to:

- read the retrieved passages and **synthesize** a coherent, well-structured answer;
- **quote and cite** accurately from the supplied text;
- handle conversation flow (follow-up questions, clarifications, the user's language);
- **refuse** to answer when the retrieved context doesn't contain the answer.

> This division of labor is the core design principle: **retrieval provides truth, the LLM provides fluency**. It is what makes the system trustworthy enough for a religious text where fabricated quotes would be unacceptable — and it means the knowledge base can be corrected or extended without retraining anything.

8. API surface

Endpoint	Method	Purpose
`/api/chat`	POST	Streamed, cited answer (SSE: sources → tokens → done); rate-limited 10/min/IP
`/api/search?q=&k=`	GET	Raw hybrid search — returns ranked passages without LLM involvement
`/api/passage/{type}/{n}`	GET	Full original text of any sermon/letter/saying (powers the citation viewer)
`/api/health`	GET	Corpus size, index size, LLM reachability — drives the UI status dot
`/`	GET	The chat frontend (static files served by FastAPI)
`/docs`	GET	Auto-generated interactive Swagger documentation

9. Frontend

A single-page chat interface (`frontend/`) written in plain HTML/CSS/JS — no framework, no build step, no CDN calls. It renders the streamed answer live, converts `[Sermon 27]` -style citations into clickable chips, shows collapsible source cards under each answer, and opens a modal with the full passage when a citation is clicked. It adapts to the OS light/dark theme and is responsive down to phone widths. Conversation history is kept client-side and trimmed to the last 20 messages.

10. Deployment & operations

- **Local:** `uvicorn app.main:app --port 8000` inside the project's virtualenv.
- **Docker:** the Dockerfile installs dependencies, runs the entire data pipeline at *build* time (so the image ships with the index — instant cold starts, no runtime downloads except nothing), runs as non-root UID 1000, and honors the host's `PORT` variable.
- **docker-compose:** app + optional Ollama sidecar for a fully local, private stack.
- **Cloud:** deployable to any Docker host; Render free tier is the current target (GitHub-connected, env vars in dashboard, sleeps when idle).
- **Security posture:** API key only in env/secrets (never in git), per-IP rate limiting, input length limits via Pydantic, CORS configurable, no server-side user data.

11. Project layout

```
app/
  main.py            FastAPI app: startup (loads retriever), CORS, static frontend
  config.py          All settings from environment / .env (pydantic-settings)
  models.py          Request/response schemas (Pydantic)
  ratelimit.py       Per-IP sliding-window rate limiter
```

```
  api/routes.py      /api/chat, /api/search, /api/passage, /api/health
  rag/
    embeddings.py    fastembed wrapper (lazy-loaded, cached model)
    vectorstore.py   ChromaDB persistent collection management
    retriever.py     hybrid search: vectors + BM25 + RRF + reference pinning
    generator.py     system prompt, LLM streaming client, extractive fallback
scripts/             download_data → scrape_alislam → build_corpus → ingest
frontend/            index.html, style.css, app.js (zero-dependency chat UI)
tests/               11 tests: corpus integrity + API behaviour
Dockerfile, docker-compose.yml, requirements.txt, .env.example, README.md
```
